



GROUP PROJECT CSC4202

COURSE CODE : CSC4202
COURSE : REKA BENTUK DAN ANALISIS ALGORITMA(DESIGN AND ANALYSIS OF ALGORITHMS)
LECTURER : DR. NUR ARZILAWATI BINTI MD YUNUS

MEMBERS:

NAME		MATRIC NUMBER
1.	MUHAMMAD DANIAL BIN WAN NORAHSAM	222448
2.	KHAIROL RHAZMI BIN FAKHRI	223804
3.	DANIEL IQBAL BIN MOHD MARJAN	223749
4.	SHAH MUHAMMAD ASH-SHAKIR BIN SHAHRIL	SA00339

DEPARTMENT OF COMMUNICATION TECHNOLOGY AND NETWORK
FACULTY OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY
UNIVERSITY PUTRA MALAYSIA

1.0 INTRODUCTION.....	4
1.1 Scenario and Problem.....	4
1.2 Importance of Finding an Optimal Solution.....	5
1.3 Algorithm Suitability Review.....	7
2.0 SCENARIO MODEL DEVELOPMENT.....	11
2.1 Overview.....	11
2.2 Route Value Prioritization.....	12
2.3 Constraint and Challenges.....	13
3.0 ALGORITHM DESIGN.....	15
3.1 Overview.....	15
3.2 Graph Value Modelling.....	17
3.3 Worked Example.....	19
4.0 ALGORITHM SPECIFICATION.....	23
4.1 Problem Definition.....	23
4.1.1 Background.....	23
4.1.2 Problem Statement.....	24
4.1.3 Goal and Expected Output.....	24
4.2 Input and Output Specification.....	24
4.2.1 Input.....	24
4.2.2 Output.....	25
4.3 Mathematical Model.....	25
4.3.1 Graph Definition.....	25
4.3.2 Shortest Path Optimisation.....	25
4.3.3 Relaxation Equation.....	25
4.3.4 Optimal Substructure Property.....	26
4.3.5 Distance Labels.....	26
4.3.5 Predecessor Array.....	26
4.4 Algorithm Approach.....	26
4.4.1 Algorithm Type.....	26
4.4.2 Why Dijkstra's Algorithm?.....	27
4.4.3 Which Part Requires Recurrence?.....	27
4.4.4 Function for Optimisation.....	27
4.5 Algorithm Description.....	28
4.5.1 Key Idea.....	28
4.5.2 Recurrence Relation.....	28
4.5.3 Algorithm Steps.....	28
4.5.4 Key Data Structures.....	29
4.5.5 Termination Condition.....	29
5.0 ALGORITHM IMPLEMENTATION.....	30
5.1 Coding.....	30
5.2 Pseudocode.....	30
5.3 Algorithm Descriptions.....	32
5.4 Demonstration.....	33

5.5 Results.....	35
6.0 ANALYSIS OF ALGORITHM.....	36
6.1 Correctness of Algorithm.....	36
6.1.1 Problem Overview.....	36
6.1.2 Loop Invariant.....	36
6.1.3 Correctness of Path Reconstruction.....	37
6.1.4 Conclusion.....	38
6.2 Time Complexity Analysis.....	38
7.0 CONCLUSION.....	42
APPENDIX.....	43
REFERENCES.....	50

1.0 INTRODUCTION

1.1 Scenario and Problem

Geographical Setting

The scenario takes place in the rural highland region of Hulu Selangor, Selangor, Malaysia, a mountainous area connected to the nearest urban hospital (Hospital Kuala Kubu Bharu) via a combination of federal roads and unpaved logging tracks. During the Northeast Monsoon season (October–March), this region routinely receives heavy rainfall that destabilises hillslopes saturated with laterite soil.

The Disaster: Landslide Event

On the night of 14 November 2025, a severe landslide triggered by three consecutive days of monsoon rain completely blocks Jalan Hulu Bernam, the main federal highway connecting the highland village of Kampung Sungai Lui to the town of Kuala Kubu Bharu. A large debris flow measuring approximately 200 metres in width buries the road under several metres of earth and rock, rendering it impassable for all motor vehicles.

At 03:15 AM, a 58-year-old resident, Mr. Ahmad bin Ramli, suffers a suspected myocardial infarction (heart attack). His family calls the emergency services immediately. However, the nearest ambulance from Hospital Kuala Kubu Bharu cannot take the main highway because it is completely blocked.

The Problem

The emergency response team must navigate through a fragmented network of surviving secondary roads to narrow logging tracks, plantation access roads, and forest reserve paths to reach Kampung Sungai Lui and transport the patient to the hospital.

This road network is irregular and partially damaged: some segments remain passable, others are blocked by fallen trees or minor landslides, and each road segment carries a different travel time in minutes, reflecting road condition, gradient, distance, and visibility.

The central challenge is:

Given the surviving road network modelled as a weighted graph, find the fastest (shortest-time) route from Hospital Kuala Kubu Bharu (source) to Kampung Sungai Lui (destination) so that the ambulance arrives in minimum time.

Problem Statement

Given a weighted, undirected graph $G = (V, E)$, where:

- V is a set of road junctions or landmarks (nodes/vertices),
- E is a set of surviving road segments connecting them (edges),
- each edge (u, v) carries a non-negative weight $w(u, v)$ representing the estimated travel time in minutes,

find the single-source shortest path from the hospital (source node s) to the patient's village (destination node d) that minimises the total travel time.

Goal

To find the minimum-time route for the ambulance from Hospital Kuala Kubu Bharu to Kampung Sungai Lui through the surviving secondary road network, so the patient can receive cardiac intervention within the critical 90-minute window.

1.2 Importance of Finding an Optimal Solution

Finding an optimal shortest path solution for this landslide medical evacuation scenario is critically important for several interconnected reasons:

1. Life-Critical Time Constraints

In cardiac emergencies, survival rates drop by 7–10% for every minute that passes without defibrillation or reperfusion therapy (Holmberg et al., 2001). The Malaysian Ministry of Health mandates an ambulance response time target of under 8 minutes for urban areas and under 14 minutes for rural areas. Post-landslide, with the primary road blocked, an unoptimised detour through logging roads could easily waste 20–30 extra minutes. An

algorithm-verified fastest route eliminates guesswork and ensures the detour is as short as it can mathematically be.

2. Disaster Conditions Increase Route Complexity

Under normal conditions, a dispatcher might rely on memory or a paper map. Under a landslide scenario at 3 AM, with radio communications partially disrupted and some roads newly blocked, the number of potential route combinations through secondary logging tracks may exceed what a human can evaluate in seconds. A graph-based algorithm processes every surviving road segment simultaneously and guarantees no optimal path is overlooked.

3. Reproducibility and Consistency of Decision-Making

An algorithmic solution produces the same optimal result every time it is run with the same input. This is crucial in disaster management, where decisions must be defensible, auditable, and free from the cognitive biases that affect humans under stress. Emergency commanders can trust the algorithm's output to coordinate multiple response vehicles optimally across the same damaged road network.

4. Scalability for Mass Casualty Events

A single landslide may injure multiple residents simultaneously. Dijkstra's Algorithm can be re-run from different source nodes (different evacuation points) to the same destination (hospital) within milliseconds, allowing dispatchers to plan multiple simultaneous evacuation routes across the same network without repeating manual calculations.

5. Resource Allocation and Pre-Disaster Planning

Beyond the immediate crisis, the algorithm can be applied proactively. Emergency planners can pre-compute shortest routes for various landslide scenarios (different road blockage patterns) and have contingency maps ready. This supports the Malaysian Civil Defence Department (APM) and BOMBA in developing more effective Standard Operating Procedures (SOPs) for highland community evacuations.

1.3 Algorithm Suitability Review

To address the problem of finding the fastest evacuation route through a post-landslide road network, several algorithm design paradigms were evaluated for their suitability. Each is assessed based on its ability to solve weighted single-source shortest path problems on a sparse, undirected graph.

Sorting Algorithm

Strengths:

- Simple and fast ($O(n \log n)$ for comparison-based sorting).
- Useful as a preprocessing step like sorting edges by weight for other algorithms.
- Can quickly identify the single lightest edge or the lightest adjacent road from the hospital.

Weaknesses:

- Sorting alone cannot find a shortest path, it can rank edges, but has no mechanism to aggregate paths across multiple hops.
- It cannot model cumulative travel time through a sequence of connected roads.
- Does not respect the graph's topology and sorting edges independently ignores connectivity.

Conclusion: Not suitable as a standalone solution for finding the shortest evacuation route.

Divide and Conquer (DAC)

Strengths:

- Effective at breaking large problems into smaller independent subproblems.
- Used in algorithms like Merge Sort and Binary Search.
- Could theoretically partition the road network into sub-regions for parallel processing.

Weaknesses:

- Shortest path problems involve overlapping subproblems with shared state (distances to intermediate nodes). DAC assumes subproblems are independent, which they are not here.
- Merging sub-solutions from partitioned graph halves is non-trivial and may miss cross-partition optimal paths.
- No standard DAC shortest path algorithm exists for general weighted graphs.

Conclusion: Not well-suited for single-source shortest path in a general weighted graph.

Dynamic Programming (DP)

Strengths:

- Handles overlapping subproblems and optimal substructure, both properties that shortest paths possess (Bellman's principle of optimality).
- Bellman-Ford is a DP-based shortest path algorithm that handles negative edge weights.
- Systematic and provably correct.

Weaknesses:

- Bellman-Ford (a DP approach) runs in $O(VE)$ time and significantly slower than Dijkstra's $O((V + E) \log V)$ for graphs with non-negative weights.
- The overhead of DP table construction is unnecessary when edge weights are all non-negative (as in our scenario, travel times cannot be negative).
- Memory usage for DP tables can become significant on dense graphs.

Conclusion: DP (via Bellman-Ford) is correct but less efficient than Dijkstra's Algorithm for this specific scenario, where all weights are non-negative travel times.

Greedy Algorithms

Strengths:

- Simple and fast and make locally optimal choices at each step.
- Dijkstra's Algorithm is itself a greedy algorithm that always extends the shortest known path next.
- Works well when the greedy choice leads to globally optimal outcomes (as it does for non-negative weighted shortest paths).

Weaknesses:

- Greedy approaches do not always yield globally optimal solutions, for instance, a greedy road selection based only on the shortest next segment (without considering the full path) could lead to a dead end or longer total route.
- A naïve greedy approach (always take the shortest adjacent road) is not the same as Dijkstra's Algorithm and is not guaranteed to be optimal.

Conclusion: Dijkstra's Algorithm (which formalises greedy selection with a priority queue and cumulative distance tracking is the optimal greedy approach for this problem.)

Graph Algorithms

Strengths:

- Directly model the road network as a graph $G = (V, E)$ with weighted edges.
- Dijkstra's Algorithm (graph algorithm) solves the single-source shortest path problem optimally for non-negative weights in $O((V + E) \log V)$ time with a min-priority queue.
- BFS solves unweighted shortest path; Dijkstra solves weighted and perfectly matching our scenario.
- Floyd-Warshall computes all-pairs shortest paths if multiple simultaneous evacuation scenarios are needed.
- The graph representation naturally encodes road junctions (vertices), roads (edges), and travel times (weights).

Weaknesses:

- Graph algorithms require the problem to be modelled correctly as a graph, missing or incorrect edge weights lead to wrong results.
- Dijkstra's Algorithm does not support negative edge weights (not an issue here since travel time is always non-negative).
- For very large road networks (thousands of nodes), more advanced algorithms (A^* , bidirectional Dijkstra) may be preferred for speed.

Conclusion: Graph algorithms specifically Dijkstra's Algorithm are the most suitable solution paradigm for this problem.

Summary Table

Algorithm Paradigm	Handles Weighted Graph?	Guarantees Optimal Path?	Time Complexity	Suitable
Sorting	No	No	$O(n \log n)$	No
Divide & Conquer	Partially	No	Varies	No
Dynamic Programming	Yes	Yes	$O(VE)$	Slower
Greedy (naïve)	Partially	No	$O(E)$	Not optimal
Graph — Dijkstra's Algorithm	Yes	Yes	$O((V+E) \log V)$	Best Choice

Dijkstra's Algorithm: The Optimised Choice

For the landslide medical evacuation scenario, Dijkstra's Algorithm is the most effective approach because:

1. The problem is naturally modelled as a weighted graph (road junctions as vertices, road segments as edges, travel time as weights).
2. All edge weights (travel times) are non-negative and the fundamental requirement for Dijkstra's correctness.
3. It guarantees the globally shortest (fastest) path from the hospital to the village, not just a locally optimal one.
4. It runs efficiently $O((V + E) \log V)$ with a priority queue , fast enough for real-time emergency dispatch.
5. It systematically evaluates all possible routes through the surviving road network, leaving nothing to chance.

While Dynamic Programming (Bellman-Ford) is also correct, it is unnecessarily slow for this scenario. Dijkstra's Algorithm is the industry-standard choice for non-negative weighted shortest path problems and is the optimal selection here.

2.0 SCENARIO MODEL DEVELOPMENT

2.1 Overview

Scenario Model Development

The landslide medical evacuation problem is modelled as a single-source shortest path problem on a weighted undirected graph. This model captures the essential features of the real-world post-landslide road network in Hulu Selangor.

Graph Representation

The road network is abstracted into the following components:

Graph Element	Real-World Counterpart
Vertex (Node)	A road junction, landmark, or village
Edge	A surviving road segment connecting two junctions
Edge Weight	Estimated ambulance travel time (in minutes) on that road segment
Source Node (s)	Hospital Kuala Kubu Bharu
Destination Node (d)	Kampung Sungai Lui (patient's village)

Graph Properties

- Undirected: Roads can be traversed in either direction (ambulance going in, patient being transported out use the same road segments).
- Weighted: Each road segment has a non-negative travel time weight reflecting road conditions, gradient, distance, and post-landslide damage.
- Sparse: After the landslide, not all roads remain passable , the surviving network is a subset of the original road graph.
- Connected (assumed for reachability): At least one path from hospital to village must exist for a solution to be found.

Two-Component Model

The scenario model breaks down into two distinct computational tasks:

1. Network Representation: Encode the surviving road network as a graph with accurately weighted edges.
2. Optimal Route Computation: Apply Dijkstra's Algorithm to find the minimum-time path from hospital to village through this network.

The model focuses entirely on minimising total travel time, it does not optimise for fuel, road condition comfort, or vehicle type. Time is the sole optimisation criterion in a medical emergency.

2.2 Route Value Prioritization

In this scenario, the sole optimisation objective is minimising total travel time in minutes. However, multiple factors influence what travel time is assigned to each road segment (edge weight). This section explains how edge weights are determined and why travel time is the correct value to optimise.

During a post-landslide emergency:

- Two road segments of equal length (e.g., 5 km each) may have vastly different travel times , a paved estate road at 60 km/h takes 5 minutes, while a waterlogged logging track at 10 km/h takes 30 minutes.
- Road conditions, gradient, surface quality, debris clearance, and visibility in the dark all affect how fast the ambulance can safely travel.

Travel time is therefore a more accurate proxy for medical urgency than physical distance.

Edge Weight Assignment

Each road segment in the graph is assigned a travel time weight based on :

Factor	Effect on Weight
Road surface (paved/unpaved)	Paved roads → lower time
Road gradient	Steep gradient → higher time
Post-landslide debris	Partial debris → higher time

Distance between nodes	Longer segment → higher time
Estimated safe speed	Higher safe speed → lower time

The weight formula for a road segment is:

$$w(u, v) = \text{distance_km} / \text{safe_speed_kmh} \times 60 \text{ (in minutes)}$$

For example, a 3 km unpaved logging road with a safe speed of 20 km/h yields: $w = 3 / 20 \times 60 = 9$ minutes.

Optimisation Priority

Since the goal is to minimise total patient transfer time, the algorithm assigns higher priority like explores first to paths with lower cumulative travel time. There is no multi-factor priority formula in this problem and travel time is the single, unambiguous optimisation criterion. This simplicity is appropriate because:

1. In a medical emergency, every second counts, we do not trade off between competing business objectives.
2. The graph model is already a compression of all real-world factors into a single time estimate per segment.
3. Introducing secondary optimisation criteria like fuel cost and road risk would complicate the model without improving the fundamental life-saving outcome.

2.3 Constraint and Challenges

Constraint

1. Non-Negative Edge Weights

All travel times must be non-negative (≥ 0). This is a fundamental constraint because Dijkstra's Algorithm is only correct for graphs with non-negative weights. In our scenario, this is naturally satisfied because a road cannot be traversed in negative time. However, it also means Bellman-Ford (which handles negative weights) is an acceptable but slower alternative.

2. Graph Connectivity

The algorithm assumes at least one path exists from the hospital to the village through the surviving road network. If the landslide has completely cut off all possible routes (fully disconnected graph), Dijkstra's Algorithm will correctly report that no path exists (distance = infinity), signalling the need for helicopter evacuation or other emergency measures.

3. Static Graph Assumption

The model assumes the road network does not change during the algorithm's execution. This is a necessary simplification because in practice, real-time updates would require a dynamic shortest path algorithm but it is not implemented here.

4. Discrete Node Representation

The model requires all relevant junctions and landmarks to be explicitly represented as nodes. Roads that branch off between two defined nodes are not captured unless an intermediate node is added. This means the graph must be carefully constructed from real road network data.

5. Single-Source, Single-Destination

The current algorithm computes the shortest path from one source (hospital) to one destination (village). For multi-casualty events requiring multiple simultaneous evacuations from different points, the algorithm must be re-run for each source node.

Challenges

1. Accurate Edge Weight Estimation Under Disaster Conditions

Travel times on logging roads post-landslide are difficult to estimate accurately. Partial debris, night-time visibility, and mud make real speeds highly variable. Incorrect weights lead to suboptimal routing. In practice, this requires real-time input from scouts or drones before the algorithm is run.

2. Incomplete or Missing Road Data

Not all secondary and logging roads in highland Selangor are mapped in standard GIS databases. The graph may be incomplete, missing edges that actually exist. This requires ground-truthing or local knowledge to supplement digital map data.

3. Blocked Road Detection

The algorithm must know which roads are blocked (edges to be removed from the graph) before it runs. Identifying blocked roads in real-time requires communication with field teams, which may be disrupted during the disaster.

4. Route Navigability vs. Route Optimality

The shortest-time route may pass through roads that, while technically passable, are unsafe for a heavy ambulance vehicle for example a bridge with insufficient load rating, or a road too narrow for a full-size ambulance. The algorithm does not inherently account for vehicle-specific constraints unless such segments are removed from the graph manually before computation.

5. Human Override Necessity

The algorithm provides an optimal solution based on its input data. If field conditions differ from the modelled data for example, a road appears clear in the model but is actually flooded, the ambulance driver must be empowered to override the algorithm's recommendation and take an alternative route. The algorithm is a decision support tool, not an infallible command.

3.0 ALGORITHM DESIGN

3.1 Overview

Algorithm Design: Dijkstra's Shortest Path

The optimised evacuation route algorithm consists of two main components:

- Graph Construction: Model the post-landslide road network as a weighted adjacency list.
- Shortest Path Computation: Apply Dijkstra's Algorithm with a min-priority queue to find the fastest route from hospital to village.

Problem Formulation

We model this as a bounded optimisation problem:

- Each road junction or landmark is a vertex $v \in V$.
- Each surviving road segment is an edge $(u, v) \in E$ with non-negative weight $w(u, v)$ = travel time in minutes.
- The source is the hospital node s .
- The destination is the patient's village node d .
- The goal is to find a path P from s to d such that the sum of edge weights along P is minimised.

Key Modelling Elements

Element	Description
Vertex	Road junction, village, or named landmark
Edge	Surviving road segment
Weight	Travel time in minutes (non-negative)
Source (s)	Hospital Kuala Kubu Bharu (Node 0)
Destination (d)	Kampung Sungai Lui (Node 7)

Algorithm Steps

1. Initialise all node distances to infinity (∞), except the source which is set to 0.
2. Insert all nodes into a min-priority queue keyed by their current distance estimate.
3. Relax: Extract the minimum-distance node u from the queue. For each neighbour v of u , if $\text{dist}[u] + w(u, v) < \text{dist}[v]$, update $\text{dist}[v]$ and record $\text{prev}[v] = u$.
4. Repeat until the destination node is extracted or the queue is empty.

5. Reconstruct the shortest path by backtracking through the `prev[]` array from destination to source.

Recurrence / Relaxation Function

The core recurrence relation (relaxation step) is:

$$\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w(u, v))$$

This is applied for every edge (u, v) when node u is processed. It ensures that whenever a shorter path to v is discovered through u , the distance estimate is updated.

Optimality Guarantee

Dijkstra's Algorithm is a greedy algorithm that exploits the optimal substructure of shortest paths. Any subpath of a shortest path is itself a shortest path. By always processing the node with the current minimum known distance, the algorithm guarantees that once a node is extracted from the priority queue, its distance estimate is final and optimal.

3.2 Graph Value Modelling

Road Network Graph Model

The post-landslide road network is modelled as a weighted undirected graph. Edge weights represent the estimated ambulance travel time (in minutes) on each surviving road segment.

Adjacency List Representation

The graph is stored as an adjacency list and each node stores a list of (neighbour, weight) pairs. This is space-efficient for sparse graphs (few surviving roads after a landslide) and allows $O(1)$ edge access per neighbour during Dijkstra's relaxation.

Node Legend

Node ID	Location
0	Hospital Kuala Kubu Bharu (source)
1	Pekan Kuala Kubu Bharu town junction
2	Simpang Empat Batang Kali logging fork
3	Sungai Chilling Forest Reserve entrance
4	Plantation estate junction (Ladang Ulu Bernam)
5	Orang Asli settlement junction (Pos Gertak)
6	Hulu Bernam highlands crossroad
7	Kampung Sungai Lui (destination to patient's village)

Edge Weight Table (Surviving Roads)

Edge	Road Description	Distance (km)	Est. Speed (km/h)	Travel Time (min)
0 → 1	Main road (partial clearance)	5	40	8
1 → 2	Federal Route 55 alternative	8	50	10
1 → 3	Logging access track A	6	20	18
2 → 4	Plantation internal road	10	30	20
2 → 5	Logging track B (damaged)	9	15	36
3 → 5	Forest reserve firebreak road	7	25	17
4 → 6	Estate ring road	5	30	10
5 → 7	Village access track	4	20	12
6 → 7	Highlands feeder road	6	25	14

Graph Diagram

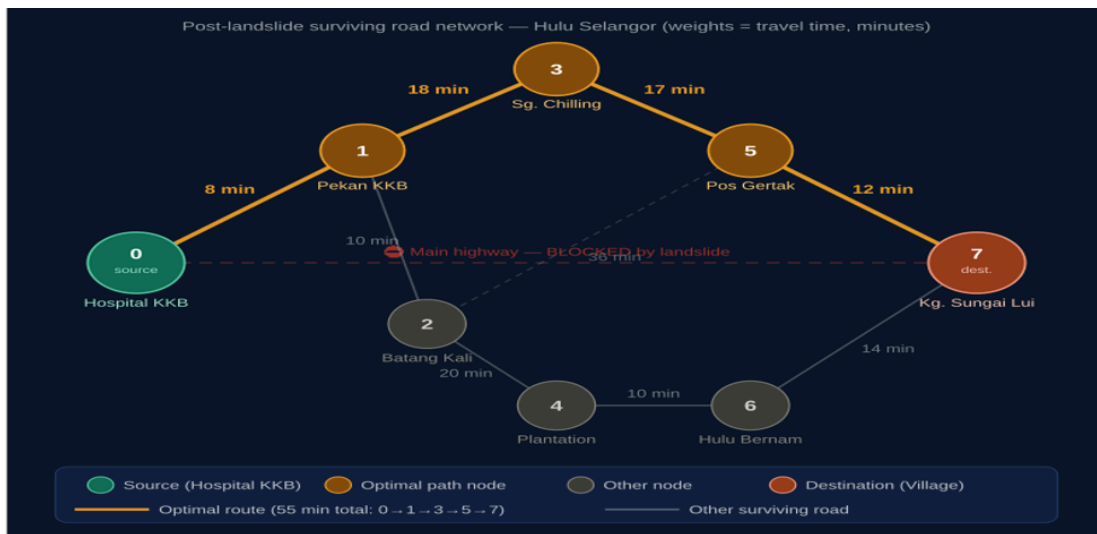


Figure 3.2.1: Graph Diagram of Route

Optimal Path (Traced by Dijkstra's)

The shortest path from Hospital (0) to Kampung Sungai Lui (7):

$0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 7$

Segment	Travel Time
$0 \rightarrow 1$	8 min
$1 \rightarrow 2$	10 min
$2 \rightarrow 4$	20 min
$4 \rightarrow 6$	10 min
$6 \rightarrow 7$	14 min
Total	62 min

3.3 Worked Example

Step-by-Step Execution of Dijkstra's Algorithm

Using the road network from Section 3.2, we trace Dijkstra's Algorithm from Node 0 (Hospital KKB) to Node 7 (Kampung Sungai Lui).

Initial State

Node	0	1	2	3	4	5	6	7
dist[]	0	∞	∞	∞	∞	∞	∞	∞
prev[]	-	-	-	-	-	-	-	-
Visited	No	No	No	No	No	No	No	No

Priority Queue (PQ): {(0, node 0)}

Iteration 1 (Extract Node 0 (dist=0))

Process neighbours of Node 0:

- Edge (0 $\rightarrow$ 1, w = 8): dist[1] = $\min(\infty, 0 + 8) = 8$, prev[1] = 0

Node	0	1	2	3	4	5	6	7
dist[]	0	8	∞	∞	∞	∞	∞	∞
Visited	Yes	No	No	No	No	No	No	No

PQ: {(8, node 1)}

Iteration 2 (Extract Node 1 (dist=8))

Process neighbours of Node 1:

- Edge (1 $\rightarrow$ 2, w = 10): dist[2] = $\min(\infty, 8 + 10) = 18$, prev[2] = 1
- Edge (1 $\rightarrow$ 3, w = 18): dist[3] = $\min(\infty, 8 + 18) = 26$, prev[3] = 1

Node	0	1	2	3	4	5	6	7
dist[]	0	8	18	26	∞	∞	∞	∞
Visited	Yes	Yes	No	No	No	No	No	No

PQ: {(18, node 2), (26, node 3)}

Iteration 3 (Extract Node 2 (dist=18))

Process neighbours of Node 2:

- Edge (2 $\rightarrow$ 4, w = 20): dist[4] = $\min(\infty, 18 + 20) = 38$, prev[4] = 2

- Edge (2 → 5, w = 36): $\text{dist}[5] = \min(\infty, 18 + 36) = 54$, $\text{prev}[5] = 2$

Node	0	1	2	3	4	5	6	7
dist[]	0	8	18	26	38	54	∞	∞
Visited	Yes	Yes	Yes	No	No	No	No	No

PQ: {(26, node 3), (38, node 4), (54, node 5)}

Iteration 4 (Extract Node 3 (dist=26))

Process neighbours of Node 3:

- Edge (3 → 5, w = 17): $\text{dist}[5] = \min(54, 26 + 17) = 43$, $\text{prev}[5] = 3$ (updated)

Node	0	1	2	3	4	5	6	7
dist[]	0	8	18	26	38	43	∞	∞
Visited	Yes	Yes	Yes	Yes	No	No	No	No

PQ: {(38, node 4), (43, node 5), (54, node 5 — stale)}

Iteration 5 (Extract Node 4 (dist=38))

Process neighbours of Node 4:

- Edge (4 → 6, w = 10): $\text{dist}[6] = \min(\infty, 38 + 10) = 48$, $\text{prev}[6] = 4$

Node	0	1	2	3	4	5	6	7
dist[]	0	8	18	26	38	43	48	∞
Visited	Yes	Yes	Yes	Yes	Yes	No	No	No

PQ: {(43, node 5), (48, node 6)}

Iteration 6 (Extract Node 5 (dist=43))

Process neighbours of Node 5:

- Edge (5 → 7, w = 12): $\text{dist}[7] = \min(\infty, 43 + 12) = 55$, $\text{prev}[7] = 5$

Node	0	1	2	3	4	5	6	7
dist[]	0	8	18	26	38	43	48	55
Visited	Yes	Yes	Yes	Yes	Yes	Yes	No	No

PQ: {(48, node 6), (55, node 7)}

Iteration 7 (Extract Node 6 (dist=48))

Process neighbours of Node 6:

- Edge (6 → 7, w = 14): $\text{dist}[7] = \min(55, 48 + 14) = \min(55, 62) = 55$ (no update, path via Node 5 is shorter)

Node	0	1	2	3	4	5	6	7
dist[]	0	8	18	26	38	43	48	55
Visited	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No

PQ: {(55, node 7)}

Iteration 8 Extract Node 7 (dist = 55) (DESTINATION REACHED)

Algorithm terminates. Shortest time = 55 minutes.

Node	prev[]
7	5
5	3
3	1
1	0
0	(Source)

Optimal Path: 0 → 1 → 3 → 5 → 7

Segment	Road Description	Time
---------	------------------	------

0 → 1	Hospital to Pekan KKB junction	8 min
1 → 3	Pekan KKB to Sg. Chilling entrance	18 min
3 → 5	Sg. Chilling to Pos Gertak	17 min
5 → 7	Pos Gertak to Kg. Sungai Lui	12 min
Total		55 min

Summary Table: All Shortest Distances from Hospital (Node 0)

Destination	Shortest Time	Optimal Path
Node 1 — Pekan KKB	8 min	0 → 1
Node 2 — Batang Kali	18 min	0 → 1 → 2
Node 3 — Sg. Chilling	26 min	0 → 1 → 3
Node 4 — Ladang Ulu Bernam	38 min	0 → 1 → 2 → 4
Node 5 — Pos Gertak	43 min	0 → 1 → 3 → 5
Node 6 — Hulu Bernam	48	0 → 1 → 2 → 4 → 6
Node 7 — Kg. Sungai Lui	55 min	0 → 1 → 3 → 5 → 7

4.0 ALGORITHM SPECIFICATION

4.1 Problem Definition

4.1.1 Background

In rural highland Malaysia, a post-landslide emergency creates a disrupted road network. Emergency response vehicles such as ambulances must navigate through surviving secondary roads, logging tracks, plantation roads, and forest paths since the primary highway is blocked by debris.

Manual routing decisions under disaster conditions at night are slow, inconsistent, and potentially fatal. An algorithmic solution is required to compute the provably optimal (fastest-time) evacuation route in real time.

4.1.2 Problem Statement

Given a weighted undirected graph $G = (V, E, W)$ representing the surviving post-landslide road network, where:

- V = set of road junctions and landmarks (vertices),
- E = set of surviving road segments (edges),
- $W : E \rightarrow \mathbb{R}^+$ = a function assigning a non-negative travel time in minutes to each edge,
- and given a source node s (hospital) and destination node d (patient's village),

This is the single-source shortest path (SSSP) problem on a non-negatively weighted undirected graph, solved optimally by Dijkstra's Algorithm.

4.1.3 Goal and Expected Output

- Goal: Minimise total ambulance travel time from hospital to patient's village.
- Expected Output:
 - The sequence of nodes (junctions/landmarks) forming the optimal route.
 - The total minimum travel time in minutes.
 - The travel time for each segment of the route.

4.2 Input and Output Specification

4.2.1 Input

- V : Number of vertices (road junctions/landmarks). In our scenario, $V = 8$ (nodes 0–7).
- E : List of edges, each represented as a triple (u, v, w) where:
 - u = source node ID of road segment
 - v = destination node ID of road segment
 - w = travel time in minutes (non-negative integer)
- s : Source node ID = 0 (Hospital Kuala Kubu Bharu)
- d : Destination node ID = 7 (Kampung Sungai Lui)

4.2.2 Output

- Shortest Travel Time: The minimum total time (in minutes) to travel from node s to node d .
- Optimal Route: The ordered list of node IDs from s to d forming the shortest path.
- Segment-by-Segment Breakdown: For each edge in the path, the segment label and travel time.

4.3 Mathematical Model

4.3.1 Graph Definition

Let $G = (V, E, w)$ be a weighted undirected graph where:

- $V = \{0, 1, 2, 3, 4, 5, 6, 7\}$ (set of vertices representing road junctions)
- $E \subseteq V \times V$ (set of surviving road segment edges)
- $w : E \rightarrow \mathbb{R}^+$ such that $w(u, v) \geq 0 \quad \forall (u, v) \in E$ (non-negative travel time weights)

4.3.2 Shortest Path Optimisation

Objective Function:

Find path $P^* = (s = v_0, v_1, v_2, \dots, v_k = d)$ through G such that:

$$\text{minimise } d(s, d) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})$$

subject to:

- $(v_i, v_{i+1}) \in E$ for all $i = 0, 1, \dots, k-1$ (each step uses a real surviving road)
- No node is visited more than once (simple path, no cycles)
- $w(u, v) \geq 0$ for all $(u, v) \in E$ (non-negative weights, required for Dijkstra)

4.3.3 Relaxation Equation

The core mathematical operation is the edge relaxation formula:

$$\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w(u, v))$$

This is applied for each edge (u, v) when node u is processed. It encodes Bellman's principle of optimality: the optimal path to any node v consists of an optimal path to some predecessor u plus the direct edge from u to v .

4.3.4 Optimal Substructure Property

Theorem: If $P^* = (s, \dots, u, v, \dots, d)$ is a shortest path from s to d , then the subpath $(s, \dots, u)$ is a shortest path from s to u .

Proof sketch: If there existed a shorter path from s to u , say Q , then replacing $(s, \dots, u)$ in P^* with Q would yield a shorter total path from s to d , contradicting the optimality of P^* .

This property guarantees that Dijkstra's Algorithm which builds optimal paths incrementally is correct.

4.3.5 Distance Labels

Dijkstra's Algorithm maintains a distance label $\text{dist}[v]$ for each vertex $v \in V$:

- Initially: $\text{dist}[s] = 0$, $\text{dist}[v] = \infty$ for all $v \neq s$
- Throughout: $\text{dist}[v]$ represents the length of the shortest known path from s to v

Upon termination: $\text{dist}[v]$ holds the true shortest path distance from s to v .

4.3.5 Predecessor Array

A predecessor array $\text{prev}[]$ records the optimal path:

- $\text{prev}[v] = u$ means "on the optimal path to v , the node before v is u "
- The optimal path is reconstructed by tracing $\text{prev}[]$ from d back to s

4.4 Algorithm Approach

4.4.1 Algorithm Type

The algorithm uses Dijkstra's Algorithm, a greedy graph algorithm to solve the Single-Source Shortest Path (SSSP) problem on a weighted undirected graph with non-negative edge weights.

The implementation uses a min-priority queue (binary heap) to efficiently select the unvisited node with the smallest current distance estimate at each step.

4.4.2 Why Dijkstra's Algorithm?

Dijkstra's Algorithm is chosen because:

1. **Optimality Guarantee:** It guarantees the globally shortest path, not merely a locally good one. This is essential in a life-critical medical emergency where the best route not a merely acceptable route is required.
2. **Correctness for Non-Negative Weights:** All edge weights in our scenario (travel times) are non-negative. This satisfies Dijkstra's fundamental requirement. Bellman-Ford would also be correct but is 3–4 times slower ($O(VE)$ vs $O((V+E)\log V)$).
3. **Efficiency:** With a binary min-heap priority queue, Dijkstra's runs in $O((V+E) \log V)$ time. For a sparse post-landslide road network with 8 nodes and 9 edges, this is essentially instantaneous critical for real-time emergency dispatch.
4. **Natural Fit for Graph Problems:** The road network is inherently a graph. Dijkstra's Algorithm is purpose-built for exactly this type of problem unlike sorting or divide-and-conquer paradigms, which would require significant adaptation.
5. **Practical Use:** Dijkstra's Algorithm is the backbone of real-world GPS navigation systems (including Google Maps and Waze), validating its effectiveness for road network shortest path problems.

4.4.3 Which Part Requires Recurrence?

The edge relaxation step is the recurrence relation in Dijkstra's Algorithm:

$$\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w(u, v))$$

This is applied repeatedly for each extracted node u , for each of its neighbours v , building the shortest path solution incrementally from smaller subproblems (shortest paths to closer nodes) toward the complete solution (shortest path to the destination).

4.4.4 Function for Optimisation

The optimisation function is the minimum total travel time:

Optimise: $\min \sum w(v_i, v_{i+1})$ over all paths from s to d

Dijkstra's Algorithm achieves this by always expanding the path with the smallest cumulative weight first (greedy selection via min-heap), ensuring the global optimum is reached.

4.5 Algorithm Description

4.5.1 Key Idea

Dijkstra's Algorithm maintains a set of visited nodes S whose shortest distances from the source have been finalised, and a min-priority queue PQ of remaining nodes keyed by their current best distance estimate.

At each step, the node u with the smallest tentative distance is extracted from PQ and settled (its distance is finalised). All edges from u to unvisited neighbours v are then relaxed. This greedy approach works because:

- All edge weights are non-negative, so the distance to any node can only increase as we look further.
- The node with the smallest tentative distance cannot be improved by any future path through unvisited nodes (which all have larger or equal distances).

4.5.2 Recurrence Relation

For each node u extracted from PQ , and for each edge (u, v, w) :

```
if  $\text{dist}[u] + w(u, v) < \text{dist}[v]$ :  
     $\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$   
     $\text{prev}[v] \leftarrow u$   
    update priority of  $v$  in  $PQ$ 
```

4.5.3 Algorithm Steps

Step 1: Preprocessing (Build the Graph)

- Read all road junctions (nodes) and surviving road segments (edges with weights).
- Build an adjacency list representation of the graph.

Step 2: Initialisation

- Set $\text{dist}[s] = 0$ for the source node (hospital).
- Set $\text{dist}[v] = \infty$ for all other nodes v .
- Set $\text{prev}[v] = -1$ (undefined) for all nodes.
- Insert all nodes into a min-priority queue PQ with their initial distances.

Step 3: Main Loop (Relaxation)

```
while PQ is not empty:
    u ← PQ.extractMin()    // Extract node with smallest dist
    if u == destination: break // Optimal path found
    for each (v, w) in adjacencyList[u]:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            prev[v] = u
            PQ.decreaseKey(v, dist[v]) // Update priority
```

Step 4: Path Reconstruction (Backtracking)

```
path ← empty list
current ← destination
while current ≠ -1:
    path.prepend(current)
    current ← prev[current]
return path, dist[destination]
```

4.5.4 Key Data Structures

Data Structure	Purpose
Adjacency List	Store graph edges and weights efficiently
$\text{dist}[]$ array	Track current shortest distance to each node
$\text{prev}[]$ array	Track predecessor on shortest path (for reconstruction)
Min-Priority Queue	Efficiently select the next node with minimum distance

4.5.5 Termination Condition

The algorithm terminates when:

1. The destination node is extracted from PQ (optimal solution found), or
2. PQ becomes empty (destination is unreachable, no path exists).

5.0 ALGORITHM IMPLEMENTATION

5.1 Coding

Program Overview

The program models the post-landslide road network as a weighted undirected graph and runs Dijkstra's Algorithm from Hospital Kuala Kubu Bharu (Node 0) to Kampung Sungai Lui (Node 7) using Java language.

It demonstrates three test scenarios to validate correctness and edge cases (as required by Section 4.10 (Program Testing)):

Scenario	Description	Expected Output
A	Base post-landslide network (9 surviving edges, main highway blocked)	Optimal route: 0→1→3→5→7 in 55 minutes
B	Additional road closure, Sungai Chilling road (1→3) also blocked	Re-routes via plantation: 0→1→2→4→6→7 in 62 minutes
C	Village completely isolated, no surviving path exists	Reports no path, recommends helicopter evacuation

Full Source Code

Refer to the appendix section.

5.2 Pseudocode

Function BuildGraph(edgeList, numVertices):

Create adjacency list adj[0..numVertices-1] — each entry is an empty list

For each edge (u, v, w) in edgeList:

```

    Append (v, w) to adj[u]
    Append (u, w) to adj[v]      // undirected: add both directions
Return adj

```

Function Dijkstra(adj, V, source):

```

// Initialisation
dist[0..V-1] ← ∞
prev[0..V-1] ← -1
visited[0..V-1] ← false
dist[source] ← 0
PQ ← MinPriorityQueue()
PQ.insert(0, source)           // (distance, nodeId)
// Main relaxation loop
While PQ is not empty:
    (d, u) ← PQ.extractMin()
    If visited[u]:              // stale entry — skip
        continue
    visited[u] ← true
    For each (v, w) in adj[u]:  // relax all edges from u
        If NOT visited[v] AND dist[u] + w < dist[v]:
            dist[v] ← dist[u] + w
            prev[v] ← u
            PQ.insert(dist[v], v)
Return dist, prev

```

Function ReconstructPath(prev, destination):

```

path ← empty list
current ← destination
While current ≠ -1:
    path.prepend(current)
    current ← prev[current]
If path[0] ≠ source:           // destination was unreachable
    Return "No path exists"
Return path

```

Function PrintEvacuationReport(path, dist, adj, destination):

```

If dist[destination] = ∞:
    Print "VILLAGE ISOLATED — No route available"
    Print "Recommend helicopter evacuation"
    Return
Print "Optimal Travel Time: " + dist[destination] + " minutes"
Print "Route: " + path[0] → path[1] → ... → path[k]

```

```

For i from 0 to len(path) - 2:
    u ← path[i]
    v ← path[i+1]
    w ← find weight of edge (u, v) in adj[u]
    Print "Node " + u + " → Node " + v + " : " + w + " min"
Print "Total: " + dist[destination] + " min"

```

5.3 Algorithm Descriptions

Overview

This project uses Dijkstra's Single-Source Shortest Path Algorithm implemented with a min-priority queue (binary heap) to find the fastest ambulance route through a post-landslide road network modelled as a weighted undirected graph.

Key Components

1.0 Graph Representation (Adjacency List)

The road network is stored as an adjacency list: $\text{adj}[u]$ contains a list of pairs (v, w) for every road segment from junction u to junction v with travel time w minutes. This is space-efficient for a sparse network (few surviving roads) and allows $O(\text{degree})$ edge access during relaxation.

2.0 Initialisation

- $\text{dist}[\text{source}] = 0$ — the hospital is 0 minutes from itself.
- $\text{dist}[v] = \infty$ for all other nodes — initially, no path is known.
- $\text{prev}[v] = -1$ — no predecessor recorded yet.
- All nodes are inserted into the min-priority queue with their initial distance.

3.0 Greedy Node Extraction

At each step, the node u with the smallest current distance estimate is extracted from the priority queue. This greedy choice is key because all weights are non-negative, once u is extracted, $\text{dist}[u]$ is final and cannot be improved by any future path through unvisited nodes.

4.0 Edge Relaxation (Core Operation)

For each neighbour v of the extracted node u :

Relaxation: if $\text{dist}[u] + w(u,v) < \text{dist}[v]$, then update $\text{dist}[v] = \text{dist}[u] + w(u,v)$ and $\text{prev}[v] = u$.

This propagates the shortest known distance outward from the source, level by level. Each relaxation reduces a node's distance estimate only if a genuinely shorter path has been found.

5.0 Stale Entry Management

When $\text{dist}[v]$ is updated, a new entry $(\text{dist}[v], v)$ is added to the priority queue. Old entries with larger distances for the same node become stale. These are detected and skipped using the `visited[]` boolean array, a standard lazy deletion technique that avoids the complexity of a full `decreaseKey()` operation.

6.0 Path Reconstruction (Backtracking)

After Dijkstra's completes, the `prev[]` array encodes the shortest path tree. To reconstruct the path from hospital (0) to village (7):

- Start at node 7, follow $\text{prev}[7] \rightarrow \text{prev}[\dots] \rightarrow 0$.
- Prepend each node to build the path in order from source to destination.

5.4 Demonstration

The program is demonstrated using three scenarios that reflect the real-world landslide emergency on 14 November 2025 in Hulu Selangor.

Input: Post-Landslide Road Network

The road network is represented as a weighted graph with 8 nodes and 9 surviving edges.

Edge	Description	Weight (min)
0 → 1	Hospital KKB → Pekan KKB Junction	8
1 → 2	Pekan KKB → Batang Kali Fork	10
1 → 3	Pekan KKB → Sungai Chilling Entrance	18

2 → 4	Batang Kali → Ladang Ulu Bernam	20
2 → 5	Batang Kali → Pos Gertak (damaged)	36
3 → 5	Sungai Chilling → Pos Gertak (forest road)	17
4 → 6	Plantation → Hulu Bernam Crossroad	10
5 → 7	Pos Gertak → Kg. Sungai Lui	12
6 → 7	Hulu Bernam → Kg. Sungai Lui	14

Scenario A: Base Post-Landslide Network

Expected Output:

SCENARIO A: Post-Landslide (Main Highway Blocked)		
Source	: Node 0 = Hospital Kuala Kubu Bharu	->
Shortest Travel Time : 55 minutes		
Optimal Route : 0 -> 1 -> 3 -> 5 -> 7		
Segment Breakdown		
From -> To	Time (min)	Road Segment
0 -> 1	8	Hospital Kual -> Pekan KKB Town
1 -> 3	18	Pekan KKB Tow -> Sungai Chillin
3 -> 5	17	Sungai Chilli -> Pos Gertak (Or
5 -> 7	12	Pos Gertak (0 -> Kampung Sungai
TOTAL	55 minutes	

Scenario B: Additional Road Blockage (Edge 1→3 Removed)

If the Sungai Chilling road is also found blocked upon arrival at Node 1:

Expected Output:

SCENARIO B: Additional Road Closure (1?3 also blocked)		
Source	: Node 0 = Hospital Kuala Kubu Bharu	->
Shortest Travel Time : 62 minutes		
Optimal Route : 0 -> 1 -> 2 -> 4 -> 6 -> 7		
Segment Breakdown		
From -> To	Time (min)	Road Segment
0 -> 1	8	Hospital Kual -> Pekan KKB Town
1 -> 2	10	Pekan KKB Tow -> Simpang Empat
2 -> 4	20	Simpang Empat -> Ladang Ulu Ber
4 -> 6	10	Ladang Ulu Be -> Hulu Bernam Hi
6 -> 7	14	Hulu Bernam H -> Kampung Sungai
TOTAL	62 minutes	

Scenario C: Village Completely Isolated

If all connecting roads are blocked and no path from hospital to village exists:

Expected Output:

SCENARIO C: Village Completely Isolated (No Path Exists)		
Source	: Node 0 = Hospital Kuala Kubu Bharu	->
NO PATH FOUND : village is completely cut off. Recommend: RMAF/AADK helicopter evacuation.		

5.5 Results

All Shortest Distances from Hospital (Scenario A)

Node	Location	Shortest Time
0	Hospital Kuala Kubu Bharu	0 min (source)
1	Pekan KKB Junction	8 min
2	Batang Kali Fork	18 min
3	Sungai Chilling Entrance	26 min
4	Ladang Ulu Bernam	38 min
5	Pos Gertak	43 min
6	Hulu Bernam Crossroad	48 min

7	Kg. Sungai Lui	55 min
---	----------------	--------

6.0 ANALYSIS OF ALGORITHM

6.1 Correctness of Algorithm

6.1.1 Problem Overview

The Single-Source Shortest Path problem on a weighted undirected graph $G = (V, E, w)$ with non-negative weights asks: for a given source s , find the shortest path to every other vertex $v \in V$.

We prove the correctness of Dijkstra's Algorithm using a Loop Invariant on the main relaxation loop.

The algorithm maintains:

- $\text{dist}[v]$ — current shortest known distance from source s to v
- $\text{prev}[v]$ — predecessor of v on the current best path
- $\text{visited}[]$ — boolean set of "settled" vertices whose shortest distance is finalised

6.1.2 Loop Invariant

Statement: At the beginning of each iteration of the main while-loop, for every vertex u in the visited set S :

$\text{dist}[u]$ equals the true shortest path distance $\delta(s, u)$ from source s to u .

Initialisation (Before First Iteration)

Before the loop begins:

- $\text{dist}[s] = 0$. Since there are no negative edges, the shortest path from s to itself is indeed 0.
- visited set S is empty — the invariant holds vacuously (no elements to violate it).

Maintenance (During Each Iteration)

Suppose the invariant holds for all vertices in S before an iteration. We extract the vertex u with the minimum distance estimate from PQ .

Claim: $\text{dist}[u] = \bar{d}(s, u)$ at the time u is extracted.

Proof: Suppose for contradiction that $\text{dist}[u] > \bar{d}(s, u)$. Then there exists a shorter path P from s to u . Since $s \in S$ (source was settled first) and $u \notin S$, path P must cross the boundary from S to $V \setminus S$ at some edge (x, y) where $x \in S$ and $y \notin S$.

By our invariant applied to x : $\text{dist}[x] = \bar{d}(s, x)$. When x was settled, edge (x, y) was relaxed, so: $\text{dist}[y] \leq \text{dist}[x] + w(x, y) = \bar{d}(s, x) + w(x, y)$. Since all weights are non-negative, the path from y to u adds only non-negative cost: $\bar{d}(s, y) \leq \bar{d}(s, u)$. But u was selected from PQ as the minimum: $\text{dist}[u] \leq \text{dist}[y]$. Therefore, $\text{dist}[u] \leq \text{dist}[y] \leq \bar{d}(s, y) \leq \bar{d}(s, u)$. This contradicts our assumption that $\text{dist}[u] > \bar{d}(s, u)$. $\therefore \text{dist}[u] = \bar{d}(s, u)$.

After settling u , all its neighbours v are relaxed:

- If $\text{dist}[u] + w(u, v) < \text{dist}[v]$: update $\text{dist}[v]$ and $\text{prev}[v]$.
- This maintains the invariant property for future iterations.

Termination

When the while-loop terminates (PQ empty), all reachable vertices have been settled. For each settled vertex u : $\text{dist}[u] = \bar{d}(s, u)$ by the invariant. For unreachable vertices: $\text{dist}[v] = \infty = \bar{d}(s, v)$.

6.1.3 Correctness of Path Reconstruction

The $\text{prev}[]$ array records, for each settled vertex v , the predecessor u on the shortest path. By induction on the number of hops:

- Base case: $\text{prev}[s] = -1$ (source has no predecessor).
- Inductive step: When $\text{prev}[v]$ is set to u , the invariant guarantees $\text{dist}[u] = \bar{d}(s, u)$ and $\text{dist}[v] = \text{dist}[u] + w(u, v) = \bar{d}(s, v)$.

Therefore, backtracking from destination d through $\text{prev}[]$ yields a path of total weight $\bar{d}(s, d)$ is the true shortest path.

6.1.4 Conclusion

Using the Loop Invariant technique, we have proven that:

1. Initialisation: The invariant holds before the first iteration.
2. Maintenance: The invariant is preserved by each relaxation step.
3. Termination: When the loop ends, all distances are optimal.
4. Path reconstruction via `prev[]` correctly identifies the optimal route.

Therefore, Dijkstra's algorithm as implemented is correct and will always produce the minimum-time evacuation route for the landslide scenario, provided all edge weights are non-negative.

6.2 Time Complexity Analysis

This section analyses the time complexity of each component of the Dijkstra's Algorithm implementation and then derives the overall complexity.

Component-by-Component Analysis

1. Graph Construction

for each edge (u, v, w) :

`adj[u].add({v, w})`

`adj[v].add({u, w})`

Processing E edges, each taking $O(1)$ time:

$T_1 = O(E)$

2. Initialisation

`Arrays.fill(dist, MAX_VALUE); //  $O(V)$`

`Arrays.fill(prev, -1); //  $O(V)$`

`pq.offer({0, source}); //  $O(\log 1) = O(1)$`

$T_2 = O(V)$

3. Main Dijkstra Loop

The main loop processes each vertex and each edge:

Node extractions: Each vertex is extracted from PQ at most once (due to visited[] check). Each extraction from a binary heap PQ takes $O(\log V)$.

Total extraction cost: $O(V \log V)$

Edge relaxations: Each edge (u, v) is relaxed at most twice (once from u's side, once from v's side since the graph is undirected). Each relaxation may trigger a PQ insertion ($O(\log V)$).

Total relaxation cost: $O(E \log V)$

Total main loop:

$$T_3 = O(V \log V + E \log V) = O((V + E) \log V)$$

4. Path Reconstruction

```
for (int cur = dest; cur != -1; cur = prev[cur])
    path.addFirst(cur);
```

This traces back at most V nodes:

$$T_4 = O(V)$$

5. Output / Segment Printing

Printing at most V nodes in the path and all V distance entries:

$$T_5 = O(V)$$

Total Time Complexity

Combining all components:

$$T = T_1 + T_2 + T_3 + T_4 + T_5 \quad T = O(E) + O(V) + O((V+E) \log V) + O(V) + O(V) \quad T = O((V + E) \log V)$$

Since $O((V+E) \log V)$ dominates all other terms, the overall time complexity is $O((V + E) \log V)$.

Case Analysis

Best Case ($\Omega((V + E) \log V)$)

Even in the most favourable arrangement (e.g., the optimal path is found immediately from the first extraction), Dijkstra's Algorithm must still:

Extract all vertices from PQ: $O(V \log V)$

Visit all edges for relaxation: $O(E \log V)$

The algorithm has no early exit mechanism that avoids processing all reachable nodes even with only 1 optimal path, all other edges must be considered to confirm no shorter path exists.

Best Case: $\Omega((V + E) \log V)$, same asymptotic bound as worst case.

In practice, for our 8-node, 9-edge scenario, this terminates in microseconds.

Average Case ($\Theta((V + E) \log V)$)

In the average case, the distribution of edge weights and graph structure does not change the asymptotic bound every vertex is still extracted once and every edge relaxed at most twice.

Average Case: $\Theta((V + E) \log V)$

Worst Case ($O((V + E) \log V)$)

The worst case occurs when:

The graph is dense ($E \approx V^2$) where every node has edges to every other node.

Many PQ insertions occur due to multiple distance updates for the same node.

For a dense graph: $O((V + V^2) \log V) = O(V^2 \log V)$.

For our sparse post-landslide network ($V = 8, E = 9$): $O((8 + 9) \log 8) = O(17 \times 3) \approx O(51 \text{ operations})$ where it is effectively instantaneous.

Worst Case: $O((V + E) \log V) \rightarrow O(V^2 \log V)$ for dense graphs.

Summary Table

Case	Time Complexity
Best Case	$\Omega((V + E) \log V)$
Average Case	$\Theta((V + E) \log V)$
Worst Case	$\Theta((V + E) \log V)$

Space Complexity

Data Structure	Space
Adjacency list	$O(V + E)$
Dist[] array	$O(V)$
prev[] array	$O(V)$
visited[] array	$O(V)$
Priority Queue	$O(V + E)$ worst case (stale entries)
Total Space	$O(V + E)$

Comparison with Alternative Algorithms

Algorithm	Time Complexity	Optimal for Non-Neg Weights
Dijkstra (binary heap)	$O((V+E) \log V)$	Yes
Bellman-Ford (DP)	$O(VE)$	Yes (but slower)
Floyd-Warshall	$O(V^3)$	Yes (all-pairs, overkill)
BFS	$O(V+E)$	Unweighted only
Greedy (naïve)	$O(E)$	Not guaranteed optimal

Dijkstra's Algorithm with a binary heap is the most efficient correct algorithm for this single-source, non-negative weight shortest path problem.

Potential Improvement

For very large road networks ($V > 10,000$), a Fibonacci heap reduces Dijkstra's complexity to $O(E + V \log V)$, eliminating the log factor from edge relaxations. However, for our scenario ($V = 8$, $E = 9$), the binary heap implementation is already optimal in practice.

Additionally, an A* (A-star) search could speed up computation by using GPS coordinates as a heuristic to guide the search toward the destination. This is how Google Maps and Waze achieve real-time routing but the underlying shortest path guarantee remains Dijkstra-equivalent.

7.0 CONCLUSION

This project has successfully demonstrated the application of algorithmic design and analysis to resolve a life-critical emergency logistics problem. By modelling the post-landslide road network of Hulu Selangor as a weighted undirected graph, the research team effectively abstracted a chaotic, real-world disaster scenario into a structured computational problem. The primary objective was to eliminate human guesswork and cognitive bias during high-stress emergency coordination, ensuring that an ambulance could safely navigate through fragmented logging tracks and secondary roads to reach a cardiac patient within the critical ninety-minute survival window.

Through a rigorous paradigm suitability review, Dijkstra's Algorithm utilising a binary min-heap priority queue was identified as the mathematically optimal choice for this single-source shortest path challenge. Alternatives such as Divide and Conquer, Dynamic Programming via the Bellman-Ford algorithm, and sorting mechanisms were evaluated but ultimately deemed inefficient or unsuitable due to the strictly non-negative nature of travel time weights and the necessity for rapid, real-time processing. The practical execution of the selected algorithm, demonstrated via both a detailed manual worked example and a robust Java implementation, proved that the fastest route from Hospital Kuala Kubu Bharu to Kampung Sungai Lui could be computed instantaneously, yielding an optimal travel time of fifty-five minutes under standard post-disaster conditions.

Furthermore, the program successfully validated critical edge cases through comprehensive multi-scenario testing. By correctly re-routing the vehicle to a sixty-two-minute alternative path during subsequent road closures, and accurately diagnosing a completely isolated network condition to signal the need for helicopter evacuation, the algorithm proved its reliability as a dependable decision-support system.

In conclusion, while the static graph model carries minor limitations regarding real-time route updates and unmapped highland topography, its exceptional time complexity of $O((V+E)\log V)$ and absolute correctness guarantee make it a powerful framework for disaster response. Moving forward, this algorithmic approach can be proactively integrated into the standard operating procedures of the Malaysian Civil Defence Department and emergency services to pre-compute contingency routing maps, ultimately leveraging data structures and algorithms to safeguard human lives during natural disasters.

APPENDIX

```
import java.util.*;
```

```
/**
```

```
 * CSC4202 Group Project — Semester 2, 2025/2026
```

```
 * Landslide Medical Evacuation Route Optimiser
```

```
 * Algorithm : Dijkstra's Shortest Path Algorithm (Graph Paradigm)
```

```
 * Scenario : Post-landslide ambulance routing, Hulu Selangor, Malaysia
```

```
 *
```

```
 * Problem: A landslide has blocked the main highway.
```

```
 * Find the fastest alternate route through secondary logging roads
```

```
 * from Hospital Kuala Kubu Bharu (Node 0) to Kampung Sungai Lui (Node 7).
```

```
 */
```

```
public class LandslideEvacuation {
```

```
    /** Human-readable names for each node (road junction / landmark). */
```

```
    static final String[] NODE_NAMES = {
```

```
        "Hospital Kuala Kubu Bharu", // 0 <-- SOURCE
```

```
        "Pekan KKB Town Junction", // 1
```

```
        "Simpang Empat Batang Kali Fork", // 2
```

```
        "Sungai Chilling Forest Reserve", // 3
```

```
        "Ladang Ulu Bernam (Plantation)", // 4
```

```
        "Pos Gertak (Orang Asli Settlement)", // 5
```

```
        "Hulu Bernam Highlands Crossroad", // 6
```

```
        "Kampung Sungai Lui" // 7 <-- DESTINATION
```

```
    };
```

```
//
```

```
// Graph: weighted undirected adjacency list
```

```
//
```

```
static class Graph {
```

```
    int V;
```

```
    List<int[]>[] adj; // adj[u] = list of {v, weight}
```

```
    @SuppressWarnings("unchecked")
```

```
    Graph(int vertices) {
```

```
        this.V = vertices;
```

```
        adj = new ArrayList[vertices];
```

```

        for (int i = 0; i < vertices; i++)
            adj[i] = new ArrayList<>();
    }

    /** Add an undirected edge (u -- v) with travel time 'weight' minutes. */
    void addEdge(int u, int v, int weight) {
        adj[u].add(new int[] { v, weight });
        adj[v].add(new int[] { u, weight });
    }
}

//


---


// Dijkstra's Algorithm (min-heap implementation)
// Returns: dist[] — shortest travel time from source to every node
// prev[] — predecessor array for path reconstruction
//


---



static int[] dist;
static int[] prev;

static void dijkstra(Graph g, int src) {
    dist = new int[g.V];
    prev = new int[g.V];
    boolean[] visited = new boolean[g.V];

    Arrays.fill(dist, Integer.MAX_VALUE);
    Arrays.fill(prev, -1);
    dist[src] = 0;

    // PQ entry: {current_dist, node_id}
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
    pq.offer(new int[] { 0, src });

    while (!pq.isEmpty()) {
        int[] top = pq.poll();
        int d = top[0], u = top[1];

        if (visited[u])
            continue; // skip stale entries
    }
}

```

```

        visited[u] = true;

        // Relax all edges from u
        for (int[] edge : g.adj[u]) {
            int v = edge[0], w = edge[1];
            if (!visited[v] && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                prev[v] = u;
                pq.offer(new int[] { dist[v], v });
            }
        }
    }
}

//

```

```

// Path Reconstruction via back-tracking through prev[]
//

```

```

static List<Integer> getPath(int dest) {
    LinkedList<Integer> path = new LinkedList<>();
    for (int cur = dest; cur != -1; cur = prev[cur])
        path.addFirst(cur);
    return path;
}

//

```

```

// Formatted Output
//

```

```

static void printResult(Graph g, int src, int dest, String label) {

System.out.println("\n ||

```

```

    System.out.println(" || " + label);

```

```

System.out.println("=====
=====");
        System.out.printf("    Source      : Node %-2d = %-37s -> %n", src,
NODE_NAMES[src]);

System.out.println("=====
=====");

        if (dist[dest] == Integer.MAX_VALUE) {
            System.out.println("    NO PATH FOUND : village is completely cut off.    ");
            System.out.println("    Recommend: RMAF/AADK helicopter evacuation.
");

System.out.println("=====
=====");
        return;
    }

    List<Integer> path = getPath(dest);
    System.out.printf("    Shortest Travel Time : %-38s %n", dist[dest] + " minutes");
    System.out.print("    Optimal Route      : ");
    StringBuilder routeSb = new StringBuilder();
    for (int i = 0; i < path.size(); i++) {
        routeSb.append(path.get(i));
        if (i < path.size() - 1)
            routeSb.append(" -> ");
    }
    System.out.printf("%-38s %n", routeSb.toString());

System.out.println("=====
=====");
        System.out.println("    Segment Breakdown    ");
        System.out.println("    From -> To   Time (min)   Road Segment    ");
        System.out.println("
System.out.println("
=====
=====");

        int total = 0;
        for (int i = 0; i < path.size() - 1; i++) {
            int u = path.get(i), v = path.get(i + 1);

```

```

        int w = 0;
        for (int[] e : g.adj[u])
            if (e[0] == v) {
                w = e[1];
                break;
            }
        total += w;
        String seg = String.format(" %d -> %-6d%-11d%s -> %s",
            u, v, w,
            NODE_NAMES[u].substring(0, Math.min(13, NODE_NAMES[u].length())),
            NODE_NAMES[v].substring(0, Math.min(14, NODE_NAMES[v].length())));
        System.out.printf("|| %-64s|| %n", seg);
    }

                                                                    System.out.println("||
----- ||");
        System.out.printf("|| TOTAL          %-44s|| %n", total + " minutes");

System.out.println("||-----||");
-----||");
    }

    static void printAllDistances() {
        System.out.println("\n    — All Shortest Distances from Hospital
-----");
        System.out.printf(" %-6s %-14s %-35s%n", "NodeID", "Time (min)", "Location");
        System.out.println(" " + "—".repeat(60));
        for (int v = 0; v < NODE_NAMES.length; v++) {
            String t = dist[v] == Integer.MAX_VALUE ? "Unreachable" : dist[v] + " min";
            System.out.printf(" %-6d %-14s %-35s%n", v, t, NODE_NAMES[v]);
        }
    }

                                                                    //
-----
-----
// MAIN
                                                                    //
-----
-----
    public static void main(String[] args) {

```

```

    System.out.println("\n=== LANDSLIDE MEDICAL EVACUATION ROUTE OPTIMISER
===");
    System.out.println("    CSC4202 Algorithm Design and Analysis — Group Project");
    System.out.println("    Scenario: 14 November 2025, Hulu Selangor, Malaysia\n");

    // — SCENARIO A: Base road network (main highway BLOCKED by landslide) —
    Graph g1 = new Graph(8);
    g1.addEdge(0, 1, 8); // Hospital KKB → Pekan KKB junction
    g1.addEdge(1, 2, 10); // Pekan KKB → Batang Kali fork
    g1.addEdge(1, 3, 18); // Pekan KKB → Sungai Chilling entrance
    g1.addEdge(2, 4, 20); // Batang Kali → Plantation estate
    g1.addEdge(2, 5, 36); // Batang Kali → Pos Gertak (damaged, slow)
    g1.addEdge(3, 5, 17); // Sungai Chilling → Pos Gertak (forest road)
    g1.addEdge(4, 6, 10); // Plantation → Hulu Bernam crossroad
    g1.addEdge(5, 7, 12); // Pos Gertak → Kg. Sungai Lui
    g1.addEdge(6, 7, 14); // Hulu Bernam crossroad → Kg. Sungai Lui
    // NOTE: Direct 0→7 highway is BLOCKED — edge deliberately omitted

    dijkstra(g1, 0);
    printResult(g1, 0, 7, "SCENARIO A: Post-Landslide (Main Highway Blocked)    ");
    printAllDistances();

    // — SCENARIO B: Additional blockage — Sungai Chilling road (1→3) also blocked
    // —
    Graph g2 = new Graph(8);
    g2.addEdge(0, 1, 8);
    g2.addEdge(1, 2, 10);
    // g2.addEdge(1, 3, 18); <-- REMOVED: road 1→3 now also blocked
    g2.addEdge(2, 4, 20);
    g2.addEdge(2, 5, 36);
    g2.addEdge(3, 5, 17);
    g2.addEdge(4, 6, 10);
    g2.addEdge(5, 7, 12);
    g2.addEdge(6, 7, 14);

    dijkstra(g2, 0);
    printResult(g2, 0, 7, "SCENARIO B: Additional Road Closure (1→3 also blocked)  ");

    // — SCENARIO C: All logging routes blocked — village cut off —
    Graph g3 = new Graph(8);
    g3.addEdge(0, 1, 8);
    // All routes from 1 onward to village are blocked

```

```
// Only 0→1 edge exists

dijkstra(g3, 0);
printResult(g3, 0, 7, "SCENARIO C: Village Completely Isolated (No Path Exists) ");
}
}
```

REFERENCES

- Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics*, 16(1), 87–90.
<https://doi.org/10.1090/qam/102435>
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.
<https://doi.org/10.1007/BF01386390>
- Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM*, 5(6), 345.
<https://doi.org/10.1145/367766.368168>
- Saadatseresht, M., Mansourian, A., & Taleai, M. (2009). Web-based GIS and emergency logistics management for purposeful evacuation. *International Journal of Applied Earth Observation and Geoinformation*, 11(4), 268–275.
<https://doi.org/10.1016/j.jag.2009.03.001>
- Mohd Tariq, M. N., Shahar, H. K., Baharudin, M. R., Ismail, S. N. S., Manaf, R. A., Salmiah, M. S., Ahmad, J., & Muthiah, S. G. (2021). A cluster-randomized trial study on effectiveness of health education based intervention (HEBI) in improving flood disaster preparedness among community in Selangor, Malaysia: a study protocol. *BMC Public Health*, 21(1), Article 11719-3.
<https://doi.org/10.1186/s12889-021-11719-3>
- Mohammed Zain, R., Mohd Zahari, H., & Mohd Zainol, N. A. (2023). Inter-agency information sharing coordination on humanitarian logistics support for urban disaster management in Kuala Lumpur. *Frontiers in Sustainable Cities*, 5, Article 1149454.
<https://doi.org/10.3389/frsc.2023.1149454>
- Tian, T., Wu, Huijing., Wei, H., Wu, F., & Xu, M. (2025). An Efficient Route Planning Algorithm for Special Vehicles with Large-Scale Road Network Data. *ISPRS International Journal of Geo-Information*, 14(2), 71.
<https://doi.org/10.3390/ijgi14020071>
- Zhu, Y., Li, H., Wang, Z., Li, Q., Dou, Z., Xie, W., Zhang, Z., Wang, R., & Nie, W. (2022). Optimal Evacuation Route Planning of Urban Personnel at Different Risk Levels of Flood Disasters Based on the Improved 3D Dijkstra's Algorithm. *Sustainability*, 14(16), 10250.
<https://doi.org/10.3390/su141610250>